

Interfacing Python with the SQLite Transactional Database Engine

SQLite is the world's most widely installed database engine, deployed in application software such as browsers for local or client data storage. Being free and open source, it offers users the liberty to modify the source code according to their needs, and has the benefits of support from a large community, portability and zero licensing costs. This article outlines how to interface Python with this popular database engine.



The role of back-end databases is very important in today's scenario of cloud or desktop based applications. Without the integration of an effective and high performance database engine, the software suite can behave abnormally or can crash. The database engines or database management systems always need to be considered when developing any real world application because the overall data and transactions are stored in the back-end database. The main role of the database engine is to provide an effective mechanism to create, manipulate, store and retrieve records from back-end database tables without any delay.

A number of database applications are available in the software market but open source database systems are becoming very popular because of the cost factor, their portability, licence-free distribution and multi-featured aspects that come without any complexities.

SQLite: A transactional database engine for assorted applications

SQLite is a powerful and multi-featured database engine, which is the most

used in the world. Billions of SQLite installations have taken place in different regions for numerous applications. The latest release of SQLite is Version 3.28.0.

SQLite possesses assorted features and computer properties including speed, compactness, being self-contained, and many others, which make this database engine quite unique and effectual as compared to other traditional database systems. SQLite is cross-platform and stable, with backward compatibility. The official portal of SQLite asserts that its developers pledge to maintain the existing properties and power of the database engine till 2050.

The source code of SQLite is available in the public domain at <https://sqlite.org/src> and is free for everyone. It can be used for any type of real world application. The complete code of SQLite is original – it has been written without taking any source code from other unknown sources.

SQLite provides the features of serverless and self-contained database engines in which there is zero-configuration with powers of transactional SQL. It is also used as

the embedded database engine for different applications; many hardware applications are deployed using SQLite.

SQLite is popular because the in-process library implements the high performance database applications without any overheads, and does not require specialised hardware. The database created using SQLite is lightweight (in the *.db* file extension) and can be ported to any other location without specific requirements like export or import. The SQLite database system is fully ACID compliant and it allows safe access to integrity and consistency based computing.

Scenarios and case studies of SQLite

SQLite is not only used for the small applications but is also widely implemented by corporate giants (Table 1).

Interfacing Python with SQLite

The Python programming language has in-built support for the SQLite database engine. With the default installation of the Python distribution, the database engine of SQLite is automatically integrated and there is no need to install it separately. During the installation of Python using the Anaconda platform (<https://www.anaconda.com/>), SQLite3 is integrated and can be used directly.

Just as Python is a package based programming language in which a large